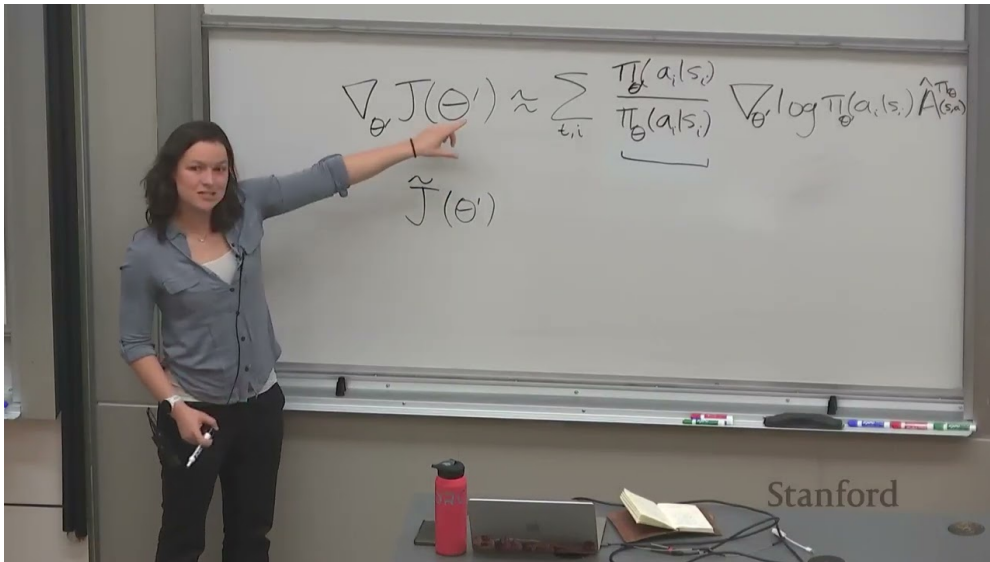


课程笔记

CS224R Lecture 5: Off-Policy Actor-Critic

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 多步梯度更新的问题	2
1.1 Importance Weights 回顾	2
1.2 本章小结	2
2 PPO: Proximal Policy Optimization	2
2.1 核心思想：限制策略偏移	2
2.2 PPO 完整算法	2
2.3 本章小结	3
3 SAC: Soft Actor-Critic	3
3.1 更彻底的 Off-Policy	3
3.2 拟合 Q 函数	3
3.3 策略更新	3
3.4 SAC 完整算法	3
3.5 本章小结	4
4 总结与延伸	4
4.1 拓展阅读	4

1 多步梯度更新的问题

On-policy actor-critic 每收集一批数据只能做一步梯度更新，效率很低。我们希望在同一批数据上做多步更新。

1.1 Importance Weights 回顾

用旧策略 π_θ 的数据来更新新策略 $\pi_{\theta'}$ 时，需要 importance weights：

$$\nabla_{\theta'} J(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \hat{A}^{\pi_\theta}(s_{t,i}, a_{t,i})$$

多步更新的风险

如果对代理目标函数（surrogate objective）做很多步梯度上升，策略会被激励大幅偏离旧策略。此时 importance weights 不再可靠，优势估计也失效，导致过拟合和性能崩塌。

1.2 本章小结

多步梯度更新能提高数据效率，但需要约束策略更新的幅度来保持稳定。

2 PPO: Proximal Policy Optimization

2.1 核心思想：限制策略偏移

两种策略：

方法 1: KL 约束

$$\max_{\theta'} \tilde{J}(\theta') \quad \text{s.t.} \quad \mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \| \pi_\theta(\cdot | s))] \leq \epsilon$$

在 LLM 的偏好优化中非常常见。

方法 2: Clipping

不直接约束策略，而是裁剪 importance weights，移除策略大幅偏离的激励：

$$\tilde{J}(\theta') = \sum_{t,i} \min \left(\frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \hat{A}^{\pi_\theta}, \text{clip} \left(\frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_\theta} \right)$$

PPO 的核心目标

这就是 PPO 的最终 surrogate objective。通过 clipping 和 min 操作，策略不再有激励大幅偏离旧策略，即使做多步更新也能保持稳定。

2.2 PPO 完整算法

1. 从当前策略 π_θ 采集一批数据
2. 拟合价值函数 $\hat{V}_\phi^\pi$
3. 计算优势 $\hat{A}^{\pi_\theta}$

4. 在这批数据上对 PPO objective 做多步梯度更新（约 3-10 步）
5. 重复

2.3 本章小结

PPO 通过 clipping importance weights 来限制策略偏移，允许在同一批数据上做多步更新，是目前最广泛使用的 on-policy RL 算法之一。

3 SAC: Soft Actor-Critic

3.1 更彻底的 Off-Policy

PPO 仍然需要定期重新收集数据。SAC 走得更远：使用 **replay buffer** 存储所有历史数据，从中随机采样来更新 Q 函数和策略。

3.2 拟合 Q 函数

SAC 直接拟合 Q^π 而不是 V^π ，使用 TD 更新：

$$\min_{\phi} \sum_{(s,a,s') \in \mathcal{D}} \left\| \hat{Q}_{\phi}^{\pi}(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} \left[\hat{Q}_{\phi}^{\pi}(s',a') \right] \right) \right\|^2$$

为什么 Q 函数可以用 off-policy 数据

Q 函数的 TD target $r(s,a) + \gamma \mathbb{E}_{a' \sim \pi} [Q(s',a')]$ 中， a' 是从**当前策略**采样的（而不是从数据中读取的）。因此 (s,a,s') 转移数据可以来自任何策略，只要 a' 是新采样的即可。

3.3 策略更新

策略优化目标加入了**熵正则化**：

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot|s)} \left[\hat{Q}^{\pi}(s,a) - \alpha \log \pi_{\theta}(a | s) \right]$$

熵项 $-\alpha \log \pi_{\theta}$ 鼓励策略保持一定的随机性，有助于探索。

3.4 SAC 完整算法

1. 用当前策略与环境交互，将转移 (s,a,r,s') 存入 replay buffer $\mathcal{B}$
2. 从 $\mathcal{B}$ 采样 mini-batch
3. 更新 Q 函数（用 TD target）
4. 更新策略（最大化 Q + 熵）
5. 重复步骤 1-4

PPO vs. SAC

- **PPO**: On-policy (加少量 off-policy), 用 clipped surrogate objective。每次收集新数据后做几步更新。更稳定、更易调参。
- **SAC**: 完全 off-policy, 用 replay buffer + Q 函数。数据效率更高。适合连续动作空间。

两者都是目前最常用的深度 RL 算法。

3.5 本章小结

SAC 通过拟合 Q 函数并使用 replay buffer 实现了完全 off-policy 的 actor-critic 算法，数据效率远高于 on-policy 方法。熵正则化鼓励探索并提高稳定性。

4 总结与延伸

1. Off-policy actor-critic 方法通过复用旧数据来提高效率。
2. **PPO**: 用 clipping 限制策略偏移，允许多步更新。简单稳定，广泛使用。
3. **SAC**: 完全 off-policy, replay buffer + Q 函数 + 熵正则化。数据效率高。
4. PPO 和 SAC 是深度 RL 中最重要的两个实用算法。

4.1 拓展阅读

- Schulman et al., *Proximal Policy Optimization Algorithms* (PPO 原文, 2017)。
- Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor* (SAC 原文, 2018)。